Лабораторная работа №15

Экран деталей объявления и кэширование изображений

О чем лабораторная?

В ленте пользователь видит только краткую информацию об объявлении. В этой работе добавим полноценный экран деталей: название, цену, описание, категорию, автора и изображение.

Изображение будем загружать с сервера при помощи библиотеки `Glide`. `Glide` автоматически использует кэш, поэтому ранее загруженная картинка не скачивается заново без необходимости.

Главное

- для одного объявления используется `GET /items/{item_id}`
- идентификатор объявления передается в Intent при переходе на экран деталей
- `photo_url` может быть равен `null`
- загрузку изображений не нужно писать вручную через `InputStream`
- `Glide` умеет показывать изображение, временную заглушку и изображение ошибки
- кэш уменьшает количество сетевых запросов и ускоряет повторное открытие экрана

Часть 1. Изучите ответ API

Откройте документацию и найдите:

`GET /items/{item_id}`

В путь передается идентификатор объявления. Сервер возвращает объект `ItemRead`:

```
{
  "id": 10,
  "title": "Гитара",
  "description": "Акустическая гитара",
  "price": 7000,
  "deal_type": "sale",
  "photo_url": "ТУТ КАКАЯ-ТО ССЫЛКА",
  "category": {
    "id": 2,
    "name": "Музыка",
    "emoji": "🎸"
  },
  "author": {
    "id": "UUID",
    "username": "student",
    "about_me": null
  }
}
```

Поле `photo_url` может содержать `null`. Приложение не должно завершаться с ошибкой, если у объявления нет изображения.

Часть 2. Добавьте запрос одного объявления

В `ApiService` добавьте метод:

```
@GET("items/{item_id}")
Call<ItemResponse> getItem(
        @Path("item_id") int itemId
);
```

Часть 3. Передайте идентификатор объявления

В адаптере при нажатии на карточку передайте `id` объявления:

```
holder.itemView.setOnClickListener(v -> {
    Intent intent = new Intent(context, DetailsActivity.class);
    intent.putExtra("item_id", item.getId());
    context.startActivity(intent);
});
```

В `DetailsActivity` получите идентификатор:

```
int itemId = getIntent().getIntExtra("item_id", -1);

if (itemId == -1) {
    finish();
    return;
}
```

Не передавайте весь объект объявления через `Intent`. Передайте только его идентификатор и загрузите актуальные данные с сервера.

Часть 4. Подключите Glide

Откройте `build.gradle.kts` модуля `app` и добавьте зависимость:

```
implementation("com.github.bumptech.glide:glide:4.16.0")
```

Часть 5. Загрузите изображение в ImageView

Добавьте в экран показа одного объявления (у меня это `activity_details.xml`) `ImageView`:

```
<ImageView
    android:id="@+id/iv_detail"
    android:layout_width="match_parent"
    android:layout_height="240dp"
    android:contentDescription="Изображение объявления"
    android:scaleType="centerCrop" />
```

Часть 6. Заполните экран деталей

В `DetailsActivity` выполните запрос:

```
ApiService api = RetrofitClient.getApiService(getApplicationContext());

api.getItem(itemId).enqueue(new Callback<ItemResponse>() {
    @Override
    public void onResponse(
            Call<ItemResponse> call,
            Response<ItemResponse> response
    ) {
        if (response.isSuccessful() && response.body() != null) {
            ItemResponse item = response.body();

            titleView.setText(item.getTitle());
            priceView.setText(String.valueOf(item.getPrice()));
            descriptionView.setText(item.getDescription());

            if (item.getCategory() != null) {
                categoryView.setText(item.getCategory().getName());
            } else {
                categoryView.setText("Категория не указана");
            }

            if (item.getAuthor() != null) {
                authorView.setText("Автор: " + item.getAuthor().getUsername());
            }

            loadImage(item.getPhotoUrl());
        } else if (response.code() == 404) {
            showMessage("Объявление не найдено");
        } else {
            showMessage("Не удалось загрузить объявление");
        }
    }

    @Override
    public void onFailure(Call<ItemResponse> call, Throwable t) {
        showMessage("Ошибка сети. Проверьте подключение");
    }
});
```

Метод `loadImage()` можно вынести отдельно:

```
private void loadImage(String imageUrl) {
    if (imageUrl == null || imageUrl.isEmpty()) {
        imageView.setImageResource(R.drawable.placeholder);
        return;
    }

    Glide.with(this)
            .load(imageUrl)
            .placeholder(R.drawable.placeholder)
            .error(R.drawable.placeholder)
            .into(imageView);
}
```

Часть 7. Самостоятельная работа

Доработайте экран деталей в своем проекте:

1. Покажите изображение, название, цену, категорию, автора и описание
2. Добавьте индикатор загрузки
3. Обработайте случаи `404`, ошибки сети и отсутствия изображения
4. Добавьте кнопку "В чат", но пока покажите при нажатии сообщение "Чат будет доступен в следующей работе"
5. Проверьте повторное открытие изображения после загрузки

Требования к итоговой версии

- нажатие на карточку открывает экран деталей
- экран запрашивает данные через `GET /items/{item_id}`
- отображаются основные данные объявления
- изображение загружается через Glide
- при отсутствии `photo_url` показывается заглушка
- экран не завершается с ошибкой при отсутствии изображения

Дополнительно (по желанию)

1. Добавьте кнопку "Обновить" для повторного запроса данных